UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Word Hunt Quest Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Word Hunt Quest

Word Hunt Quest is a standalone editable Unity game template. Drag from the first letter to the last letter of a target word. Allowed directions depend on the camp definition; later camps include diagonal and backward words. Find every required word to finish. Optional bonus words, shaped masks, near-miss decoys and timed camps add variety. Daily trails are locally generated puzzles, not a remote service.

Includes 25 curated levels, level generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Word Hunt Quest > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/WordHunt/Scenes/WordHunt_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/WordHunt`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/wordhunt contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/WhBoard.cs`
- `Scripts/WhBoosters.cs`
- `Scripts/WhLevels.cs`
- `Scripts/WhLogic.cs`
- `Scripts/WhScenery.cs`
- `Scripts/WordHuntGame.cs`
- `Scripts/WordHuntGame.Debug.cs`
- `Scripts/WordHuntGame.Ui.cs`

Main flow: WordHuntGame.cs. UI and popup consumers: WordHuntGame.Ui.cs. Model/generator/loading: WhLogic.cs, WhLevels.cs, WhBoosters.cs. Authoring: Editor/WordHuntLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag from the first letter to the last letter of a target word. Allowed directions depend on the camp definition; later camps include diagonal and backward words. Find every required word to finish. Optional bonus words, shaped masks, near-miss decoys and timed camps add variety. Daily trails are locally generated puzzles, not a remote service.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. WhBoosters.All in WhBoosters.cs owns Name, Desc, Guide, Unlock, FreeCharges and Color for every helper.

Helper	Unlock	Free charges	Action and effect
Spyglass	3	3	Highlights the first letter of an unfound word.
Torch	6	2	Lights the row, column or diagonal containing an unfound word.
Hourglass	9	2	Freezes the clock for 30 seconds. On timed camps this prevents countdown during the freeze.
Map	13	1	Reveals an entire hidden word.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. Undo needs an eligible previous move, Scoop needs sand, Spin needs a nonsymmetric comb and Drip needs a multi-cell comb. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Word Hunt Quest > Level Editor**. Exact implementation: `Editor/WordHuntLevelEditor.cs`.

Duplicate an early camp. Set Theme title, theme, Grid size 5–15, Directions, Decoys, Breather, Timed, Time limit and First-time tip. Add word or Set as bonus; select a word then drag across the editor grid to place it. Use mask presets for shaped boards. Fill seed, Regenerate fill letters, New seed + fill and Re-place all words have different effects: the last one clears fixed placements. Keep every word within the active mask. Validate, Save and Play this level.

1. Back up Resources/wordhunt/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`WordHuntLevelEditor.cs:57`	`[MenuItem("Szekipapa77/Word Hunt Quest/Level Editor")]`
`WordHuntLevelEditor.cs:100`	`if (playing) EditorGUILayout.HelpBox("The Level Editor works in Edit mode. Stop Play mode to keep editing.", MessageType.Warning);`
`WordHuntLevelEditor.cs:121`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`WordHuntLevelEditor.cs:122`	`if (GUILayout.Button("Revert", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Revert", "Discard unsaved changes?", "Revert", "Cancel"))) Load();`
`WordHuntLevelEditor.cs:123`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(90))) ValidateAll();`

WordHuntLevelEditor.cs:149	`if (GUILayout.Button("Add")) { var nd = NewLevel(_sel + 2); _file.levels.Insert(Mathf.Min(_sel + 1, _file.levels.Count), nd); _sel = Mathf.Min(_sel + 1, _file.levels.Count - 1); Dirty(); }`
WordHuntLevelEditor.cs:150	`if (GUILayout.Button("Duplicate") && Cur != null) { var c = Cur.Copy(); c.title += " copy"; _file.levels.Insert(_sel + 1, c); _sel++; Dirty(); }`
WordHuntLevelEditor.cs:151	`if (GUILayout.Button("Delete") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.title}'?", "Delete", "Cancel")) { _file.levels.RemoveAt(_sel); _sel = Mathf.Max(0, _sel - 1); Dirty(); }`
WordHuntLevelEditor.cs:155	`if (GUILayout.Button("Move Up")) { Swap(_sel, _sel - 1); _sel--; }`
WordHuntLevelEditor.cs:157	`if (GUILayout.Button("Move Down")) { Swap(_sel, _sel + 1); _sel++; }`
WordHuntLevelEditor.cs:198	`EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.boldLabel);`
WordHuntLevelEditor.cs:201	`d.title = EditorGUILayout.TextField("Theme title", d.title);`
WordHuntLevelEditor.cs:207	`int size = EditorGUILayout.IntSlider("Grid size", d.size, 5, 15);`
WordHuntLevelEditor.cs:209	`int dirs = EditorGUILayout.Popup("Directions", d.dirs, WhDirs.Names);`
WordHuntLevelEditor.cs:211	`int dec = EditorGUILayout.IntSlider(new GUIContent("Decoys (near-misses)", "Look-alike fragments of target words hidden in the filler (JAGUA, AGUAR, JAGUOR)."), d.decoys, 0, 10);`
WordHuntLevelEditor.cs:214	`d.breather = EditorGUILayout.ToggleLeft("Breather (easy rest stop)", d.breather, GUILayout.Width(180));`
WordHuntLevelEditor.cs:215	`d.timed = EditorGUILayout.ToggleLeft("Timed", d.timed, GUILayout.Width(70));`
WordHuntLevelEditor.cs:218	`d.timeLimit = EditorGUILayout.IntField("Time limit (s)", d.timeLimit);`

`WordHuntLevelEditor.cs:219`	`if (GUILayout.Button("Auto", GUILayout.Width(50))) d.timeLimit = WhGen.DefaultTimeLimit(d);`
`WordHuntLevelEditor.cs:222`	`EditorGUILayout.LabelField("First-time tip (shown when the level starts)");`
`WordHuntLevelEditor.cs:223`	`d.tip = EditorGUILayout.TextArea(d.tip ?? "", GUILayout.MinHeight(30));`
`WordHuntLevelEditor.cs:229`	`if (GUILayout.Button("Validate", GUILayout.Height(26))) { var rep = WhReport.Of(d, IsWord); _reports[_sel] = rep; d.score = rep.Score; _status = $"Level {_sel + 1}: {rep}"; }`
`WordHuntLevelEditor.cs:230`	`if (GUILayout.Button("Play this level (quick test)", GUILayout.Height(26))) PlayLevel(_sel + 1);`
`WordHuntLevelEditor.cs:233`	`else EditorGUILayout.LabelField($"par {WhGen.Par(d)} s · score {d.score:0.0}", EditorStyles.miniLabel);`
`WordHuntLevelEditor.cs:244`	`EditorGUILayout.LabelField("Words (click a word, then drag on the grid to place it)", EditorStyles.boldLabel);`
`WordHuntLevelEditor.cs:259`	`if (GUILayout.Button(new GUIContent((isBonus ? "★ " : "") + word, tip), EditorStyles.miniButtonLeft, GUILayout.Width(100))) { _placeWord = word; _tool = Tool.Place; }`
`WordHuntLevelEditor.cs:260`	`if (GUILayout.Button("x", EditorStyles.miniButtonRight, GUILayout.Width(20)))`
`WordHuntLevelEditor.cs:279`	`if (GUILayout.Button("Add word", GUILayout.Width(80))) { d.words = words.Concat(new[] { nw }).ToArray(); _newWord = ""; Dirty(); Rebuild(d, true); GUI.FocusControl(null); }`
`WordHuntLevelEditor.cs:280`	`if (GUILayout.Button("Set as bonus", GUILayout.Width(90))) { d.bonus = nw; _newWord = ""; Dirty(); Rebuild(d, true); GUI.FocusControl(null); }`

WordHuntLevelEditor.cs:289	_showSuggest = EditorGUILayout.Foldout(_showSuggest, $"Theme suggestions ({theme.Title}) — click to add", true);
WordHuntLevelEditor.cs:313	EditorGUILayout.LabelField("Grid", EditorStyles.boldLabel);
WordHuntLevelEditor.cs:321	if (GUILayout.Button(k, EditorStyles.miniButton, GUILayout.Width(62))) { d.mask = k == "full" ? new string[0] : WhMask.Preset(k, d.size); Dirty(); Rebuild(d, true); }
WordHuntLevelEditor.cs:324	d.seed = EditorGUILayout.IntField("Fill seed", d.seed, GUILayout.Width(260));
WordHuntLevelEditor.cs:325	if (GUILayout.Button("Regenerate fill letters", GUILayout.Width(150))) { Dirty(); Rebuild(d, true); }
WordHuntLevelEditor.cs:326	if (GUILayout.Button("New seed + fill", GUILayout.Width(110))) { d.seed = UnityEngine.Random.Range(1, 999999); Dirty(); Rebuild(d, true); }
WordHuntLevelEditor.cs:327	if (GUILayout.Button("Re-place all words", GUILayout.Width(130))) { d.placed = new WhPlace[0]; Dirty(); Rebuild(d, true); }
WordHuntLevelEditor.cs:331	if (p == null) { EditorGUILayout.HelpBox("Cannot build this grid: " + err + ". Remove a long word, enlarge the grid or reduce the mask.", MessageType.Error); return; }
WordHuntLevelEditor.cs:369	EditorGUILayout.LabelField($"{p.ActiveCells} letter cells · {p.Overlaps} crossings · {p.Decoys.Count} decoys · par {WhGen.Par(d)} s", EditorStyles.miniLabel);
WordHuntLevelEditor.cs:501	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (same generator as the endless levels)", true);
WordHuntLevelEditor.cs:504	_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 50);
WordHuntLevelEditor.cs:505	_genStart = EditorGUILayout.IntField("Start at level #", _genStart);

`WordHuntLevelEditor.cs:506`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
`WordHuntLevelEditor.cs:507`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1);`
`WordHuntLevelEditor.cs:508`	`_genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`WordHuntLevelEditor.cs:509`	`_genEvery = EditorGUILayout.IntSlider("Breather every N levels", _genEvery, 0, 10);`
`WordHuntLevelEditor.cs:510`	`_genDrop = EditorGUILayout.Slider("Breather drop", _genDrop, 0, 0.6f);`
`WordHuntLevelEditor.cs:511`	`_genReplace = EditorGUILayout.ToggleLeft("Replace existing levels in that range (otherwise insert)", _genReplace);`
`WordHuntLevelEditor.cs:520`	`EditorGUILayout.HelpBox($"Difficulty drives grid size (6–12), directions (across/down → diagonals → backwards), word count, decoys, shaped maps, bonus words and timed levels. Every generated level is built and validated.", MessageType.None);`
`WordHuntLevelEditor.cs:521`	`if (GUILayout.Button($"Generate {_genCount} levels", GUILayout.Height(28))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Count 1–50; Start at level #; Seed; Start/End difficulty; Breather every N levels 0–10; Breather drop 0–0.6; Replace existing levels in that range (otherwise insert). Generated difficulty controls grid size, directions, word count, decoys, masks, bonus words and timed camps. Each candidate is built and validated. Start 26 appends to the bank; insertion shifts following records.

Worked extension: back up the bank, generate five candidates after the original 25 at moderate difficulty, validate them, then Save. Reopen the JSON and confirm the count changed. An unchecked replace option inserts rather than silently destroying later data. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/WordHunt/Resources/wordhunt/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "title": "Jungle Animals",
 "size": 6,
 "dirs": 0,
 "words": [
   "JAGUAR",
   "MONKEY",
   "SLOTH",
   "PARROT"
 ],
 "bonus": "",
 "decoys": 0,
 "mask": [],
 "seed": 36,
 "breather": false,
 "timed": false,
 "timeLimit": 0,
 "tip": "Drag across the letters to circle each word in the list.",
 "placed": [
   {
     "word": "JAGUAR",
     "r": 0,
     "c": 4,
     "dr": 1,
     "dc": 0
   },
   {
     "word": "MONKEY",
     "r": 0,
     "c": 5,
     "dr": 1,
     "dc": 0
   },
   {
     "word": "PARROT",
     "r": 0,
     "c": 0,
     "dr": 1,
     "dc": 0
   },
   {
     "word": "SLOTH",
     "r": 0,
     "c": 3,
     "dr": 1,
     "dc": 0
   }
 ],
 "grid": [
   "PIESJM",
   "ASNLAO",
   "RPXOGN",
   "RRHTUK",
   "OOCHAE",
   "TOTDRY"
 ],
 "score": 0.52
}
```


Root keys: levels. Inspect WhLogic.cs, WhLevels.cs, WhBoosters.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. For Word Hunt, words, masks and placements are gameplay data; interface translation is a separate task.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/WordHuntGame.Ui.cs`. Find `Teal`, currently `#1F8A7A`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Word Hunt has a separate WhBoard.Ink for board letters; changing UI Ink alone does not recolour letters.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code

Source	Exact current code
`WhBoard.cs:16 / class/static declarations`	`HzUI.Hex("#2EC4B6"), HzUI.Hex("#FF6B6B"), HzUI.Hex("#FFC145"), HzUI.Hex("#9B5DE5"),`
`WhBoard.cs:17 / class/static declarations`	`HzUI.Hex("#4CC9F0"), HzUI.Hex("#8AC926"), HzUI.Hex("#F15BB5"), HzUI.Hex("#FF924C"),`
`WhBoard.cs:19 / class/static declarations`	`public static readonly Color Ink = HzUI.Hex("#4A3420");`
`WhBoard.cs:20 / class/static declarations`	`public static readonly Color BonusGold = HzUI.Hex("#FFD34D");`
`WhBoard.cs:50 / class/static declarations`	`var frame = HzUI.Panel("Frame", parent, HzUI.Hex("#6B4423"), 40);`
`WhBoard.cs:54 / class/static declarations`	`var inner = HzUI.Panel("FrameIn", frame.transform, HzUI.Hex("#8A5A30"), 32);`
`WhBoard.cs:59 / class/static declarations`	`var paper = HzUI.Panel("Paper", frame.transform, HzUI.Hex("#F4E6C3"), 22);`
`WhBoard.cs:65 / class/static declarations`	`var dark = HzUI.Panel("Water", frame.transform, HzUI.Hex("#3E2A17"), 22);`
`WhBoard.cs:71 / class/static declarations`	`var rv = HzUI.Panel("Rivet", frame.transform, HzUI.Hex("#E2B04A"), 10);`
`WhBoard.cs:90 / class/static declarations`	`var t = HzUI.Panel("T", tiles, HzUI.Hex("#F4E6C3"), rad);`

`WhBoard.cs:195 / public void AddBonusCapsule(bool animate)`	`var edge = HzUI.Img("Edge", img.transform, HzUI.RoundRect(64, 6), HzUI.Hex("#B8860B"));`
`WhBoard.cs:214 / public void AddRing(int r, int c, bool animate)`	`var ring = HzUI.Img("Ring", _rings, WhArt.Ring, HzUI.Hex("#E8862A"));`
`WhBoosters.cs:24 / class/static declarations`	`Color = HzUI.Hex("#1F8A7A"),`
`WhBoosters.cs:28 / class/static declarations`	`Color = HzUI.Hex("#E8862A"),`
`WhBoosters.cs:32 / class/static declarations`	`Color = HzUI.Hex("#B5442F"),`
`WhBoosters.cs:36 / class/static declarations`	`Color = HzUI.Hex("#7B4FC9"),`
`WhScenery.cs:79 / class/static declarations`	`static readonly string[] Greens = { "#14532D", "#166534", "#1F7A45", "#2D8F57", "#3FA866", "#0F3D2A", "#246B3F" };`
`WhScenery.cs:85 / class/static declarations`	`var bg = HzUI.Img("Sky", root, HzUI.Gradient(HzUI.Hex("#1B5E3F"), HzUI.Hex("#0B2E21")), Color.white);`
`WhScenery.cs:116 / class/static declarations`	`var m = HzUI.Panel("Bush", root, HzUI.Hex("#0A2A1C"), (int)(d / 2));`
`WhScenery.cs:146 / void Cluster(Vector2 at, float baseAngle, int count, float scale)`	`var vine = HzUI.Panel("Vine", root, HzUI.Hex("#2F6B3A"), 4);`
`WordHuntGame.cs:177 / void RefreshTimer(bool force)`	`if (_timeChip) _timeChip.color = Timed && _remaining <= 10 ? HzUI.Hex("#B5442F") : TimeChipColor;`
`WordHuntGame.cs:320 / void Use(WhBooster b)`	`Toast("Clock frozen for 30 s!", HzUI.Hex("#BFEFFF"), 1.3f);`
`WordHuntGame.cs:352 / void OfferRefill(WhBooster b)`	`("No thanks", HzUI.Hex("#8C7A66"), null));`
`WordHuntGame.Ui.cs:13 / class/static declarations`	`static readonly Color Ink = HzUI.Hex("#4A3420");`
`WordHuntGame.Ui.cs:14 / class/static declarations`	`static readonly Color Parch = HzUI.Hex("#F4E6C3");`
`WordHuntGame.Ui.cs:15 / class/static declarations`	`static readonly Color Orange = HzUI.Hex("#E8862A");`
`WordHuntGame.Ui.cs:16 / class/static declarations`	`static readonly Color Gold = HzUI.Hex("#F2C14E");`

`WordHuntGame.Ui.cs:17 / class/static declarations`	`static readonly Color Green = HzUI.Hex("#3DAA5C");`
`WordHuntGame.Ui.cs:18 / class/static declarations`	`static readonly Color Wood = HzUI.Hex("#6B4423");`
`WordHuntGame.Ui.cs:19 / class/static declarations`	`static readonly Color Teal = HzUI.Hex("#1F8A7A");`
`WordHuntGame.Ui.cs:20 / class/static declarations`	`static readonly Color Jungle = HzUI.Hex("#2E6B4F");`
`WordHuntGame.Ui.cs:22 / class/static declarations`	`static readonly Color FreezeColor = HzUI.Hex("#2B7FA8");`
`WordHuntGame.Ui.cs:23 / class/static declarations`	`static readonly Color BonusTextColor = HzUI.Hex("#FFE07A");`
`WordHuntGame.Ui.cs:127 / void ShowTitle()`	`var col = d == 0 ? Teal : d == 1 ? Orange : HzUI.Hex("#B5442F");`
`WordHuntGame.Ui.cs:304 / void RefreshFreeze()`	`if (_timeText) _timeText.color = _freeze > 0 ? HzUI.Hex("#DDF6FF") : Color.white;`
`WordHuntGame.Ui.cs:334 / string BonusMask()`	`var line = HzUI.Img("Line", c.Root, HzUI.RoundRect(26, 4), bonus ? BonusTextColor : HzUI.Hex("#B98A55"));`
`WordHuntGame.Ui.cs:460 / void ApplyBonusChip(bool animate)`	`_bonusChip.Bg.color = HzUI.Hex("#C99A1E");`
`WordHuntGame.Ui.cs:506 / void BuildBoosterBar(Rect r)`	`var ring = HzUI.Img("GuideRing", v.Root, HzUI.RoundRect(44, 8), HzUI.Hex("#7CFFB2")); ring.raycastTarget = false;`
`WordHuntGame.Ui.cs:721 / void HidePauseVeil()`	`var b = HzUI.Label("Body", card.transform, body, 40, HzUI.Hex("#5A4636"), TextAnchor.MiddleCenter, false);`
`WordHuntGame.Ui.cs:764 / void ShowUnlock(WhBoosterDef def, Action then)`	`string body = $"<size=52><color=#{ColorUtility.ToHtmlStringRGB(def.Color)}>{def.Name.ToUpperInvariant()}</color></size>\n{def.Desc}\n<color=#2E8B57>+{def.FreeCharges} free to start</color>";`
`WordHuntGame.Ui.cs:782 / void ShowWin(float total, int stars, bool newBest)`	`? new (string, Color, Action)[] { ("Next Camp", Green, () => StartCamp(_camp + 1)), ("Home", HzUI.Hex("#8C6A45"), GoHome) }`

`WordHuntGame.Ui.cs:783 / void ShowWin(float total, int stars, bool newBest)`	`: new (string, Color, Action)[] { ("Random Trail", Teal, () => StartDaily(_daily, true)), ("Home", HzUI.Hex("#8C6A45"), GoHome) };`
`WordHuntGame.Ui.cs:801 / void Box(string label, string value, float x, Color col)`	`var bx = HzUI.Panel("Box", d.Extra, HzUI.Hex("#1F4A36"), 26);`
`WordHuntGame.Ui.cs:810 / void Box(string label, string value, float x, Color col)`	`var nb = HzUI.Panel("NewBest", d.Extra, HzUI.Hex("#E2574C"), 22);`
`WordHuntGame.Ui.cs:819 / void Box(string label, string value, float x, Color col)`	`var bw = HzUI.Label("Bonus", d.Extra, "+ bonus word " + _p.Bonus.Text, 32, HzUI.Hex("#B8860B"));`
`WordHuntGame.Ui.cs:837 / void ShowOutOfTime()`	`list.Add(("Home", HzUI.Hex("#8C6A45"), GoHome));`
`WordHuntGame.Ui.cs:838 / void ShowOutOfTime()`	`ShowDialog("Out of time!", HzUI.Hex("#B5442F"), WhArt.Art("booster_hourglass"), $"You found {_st.FoundCount} of {_p.Words.Count} words.\nKeep searching?", list.ToArray());`
`WordHuntGame.Ui.cs:845 / void ShowSettingsDialog()`	`if (_inGame) buttons.Add(("Home", HzUI.Hex("#7B61FF"), GoHome));`
`WordHuntGame.Ui.cs:858 / void Refresh()`	`b.GetComponent<Image>().color = on ? Green : HzUI.Hex("#9A8C7A");`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/WordHuntGame.Ui.cs`, find the displayed literal `"WORD HUNT"`, and replace its displayed value with `"JUNGLE WORDS"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. WhBoosters.All in WhBoosters.cs owns Name, Desc, Guide, Unlock, FreeCharges and Color for every helper. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`WhBoard.cs:116 / class/static declarations`	`var t = HzUI.Label("C", letters, p.Grid[r, c].ToString(), fs, Ink);`
`WhBoosters.cs:23 / class/static declarations`	`new WhBoosterDef { Id = WhBooster.Spyglass, Name = "Spyglass", Art = "booster_spyglass", Icon = HzIcons.Search, Unlock = 3, FreeCharges = 3,`
`WhBoosters.cs:25 / class/static declarations`	`Desc = "Reveals the first letter of a hidden word.",`
`WhBoosters.cs:26 / class/static declarations`	`Guide = "Can't spot a word? Tap the SPYGLASS to reveal where one starts." },`
`WhBoosters.cs:27 / class/static declarations`	`new WhBoosterDef { Id = WhBooster.Torch, Name = "Torch", Art = "booster_torch", Icon = HzIcons.Bolt, Unlock = 6, FreeCharges = 2,`
`WhBoosters.cs:29 / class/static declarations`	`Desc = "Lights up the whole line (row, column or diagonal) a hidden word lies on.",`
`WhBoosters.cs:30 / class/static declarations`	`Guide = "Tap the TORCH: a word hides somewhere on the glowing line." },`
`WhBoosters.cs:31 / class/static declarations`	`new WhBoosterDef { Id = WhBooster.Hourglass, Name = "Hourglass", Art = "booster_hourglass", Icon = HzIcons.Timer, Unlock = 9, FreeCharges = 2,`

WhBoosters.cs:33 / class/static declarations	Desc = "Stops the clock for 30 seconds. On timed camps that buys you extra time.",
WhBoosters.cs:34 / class/static declarations	Guide = "Racing the clock? Tap the HOURGLASS to freeze time for 30 seconds." },
WhBoosters.cs:35 / class/static declarations	new WhBoosterDef { Id = WhBooster.Map, Name = "Map", Art = "booster_map", Icon = HzIcons.Flag, Unlock = 13, FreeCharges = 1,
WhBoosters.cs:37 / class/static declarations	Desc = "Reveals a whole word on the grid.",
WhBoosters.cs:38 / class/static declarations	Guide = "Tap the MAP to circle one hidden word for you." },
WordHuntGame.cs:65 / void LoadData()	_themes = WhContent.Parse(th ? th.text : "");
WordHuntGame.cs:212 / void MarkFound(int i, bool byClue)	else if (!byClue && _combo >= 2) Toast($"Combo x{_combo + 1}!", Gold, 0.9f);
WordHuntGame.cs:258 / void OnBooster(WhBooster b)	Toast($"Unlocks at camp {WhBoosters.Def(b).Unlock}", Color.white, 1.2f);
WordHuntGame.cs:320 / void Use(WhBooster b)	Toast("Clock frozen for 30 s!", HzUI.Hex("#BFEFFF"), 1.3f);
WordHuntGame.Ui.cs:99 / void ShowTitle()	var t1 = HzUI.Label("Title", card.transform, "WORD HUNT", 130, Jungle);
WordHuntGame.Ui.cs:105 / void ShowTitle()	var q = HzUI.Label("Quest", rib.transform, "QUEST", 64, Color.white); q.rectTransform.Stretch();
WordHuntGame.Ui.cs:110 / void ShowTitle()	var sub = HzUI.Label("Sub", _layer, "Find the hidden words. They can run in any direction!", 40, Color.white, TextAnchor.MiddleCenter, false);
WordHuntGame.Ui.cs:115 / void ShowTitle()	var play = HzUI.Button("Play", _layer, "PLAY CAMP " + camp, Green, Color.white, () => StartCamp(Level), 66, 46);
WordHuntGame.Ui.cs:121 / void ShowTitle()	var dl = HzUI.Label("DailyL", _layer, "DAILY TRAIL", 46, Gold);
WordHuntGame.Ui.cs:128 / void ShowTitle()	var b = HzUI.Button("Daily" + d, _layer, null, col, Color.white, () => StartDaily(diff, false), 40, 34);

WordHuntGame.Ui.cs:130 / void ShowTitle()	var name = HzUI.Label("N", b.transform, DiffNames[d], 46, Color.white);
WordHuntGame.Ui.cs:135 / void ShowTitle()	var info = HzUI.Label("I", b.transform, done ? "DONE " + Fmt(HzSave.GetInt($"{GameId}_dtime_{Today}_{d}", 0) / 100f) : best > 0 ? "BEST " + Fmt(best) : (d == 0 ? "9x9" : d == 1 ? "12x12" : "15x15"), 30, new Color(1, 1, 1, 0.9f));
WordHuntGame.Ui.cs:211 / void BuildGameView()	var hdr = HzUI.Label("Hdr", pill.transform, head, 48, Gold);
WordHuntGame.Ui.cs:216 / void BuildGameView()	var thm = HzUI.Label("Theme", pill.transform, sub, 32, Color.white, TextAnchor.MiddleCenter, false);
WordHuntGame.Ui.cs:265 / void BuildInfoRow(float x0, float y, float w)	_timeText = HzUI.Label("Time", _timeChip.transform, Fmt(ShownTime), 58, Color.white);
WordHuntGame.Ui.cs:274 / void BuildInfoRow(float x0, float y, float w)	_countText = HzUI.Label("C", cnt.transform, "", 50, Color.white);
WordHuntGame.Ui.cs:284 / void BuildInfoRow(float x0, float y, float w)	var bl = HzUI.Label("BL", bst.transform, label, 26, new Color(1, 1, 1, 0.65f));
WordHuntGame.Ui.cs:286 / void BuildInfoRow(float x0, float y, float w)	_bestText = HzUI.Label("BV", bst.transform, val, 46, Gold);
WordHuntGame.Ui.cs:295 / void UpdateCount(bool punch)	_countText.text = $"{_st.FoundCount}/{_p.Words.Count}";
WordHuntGame.Ui.cs:336 / string BonusMask()	c.Label = HzUI.Label("T", c.Root, text, (int)ChipBaseFont, bonus ? BonusTextColor : Ink);
WordHuntGame.Ui.cs:392 / void LayoutWordList(RectTransform root, Rect r, bool port)	var hd = HzUI.Label("Head", root, "FIND THESE WORDS", 38, Gold);
WordHuntGame.Ui.cs:479 / void BuildBoosterBar(Rect r)	var btn = HzUI.Button("Booster" + d.Name, bar.transform, null, d.Color, Color.white, () => OnBooster(id), 40, 36);
WordHuntGame.Ui.cs:490 / void BuildBoosterBar(Rect r)	var cap = HzUI.Label("Caption", v.Root, d.Name.ToUpperInvariant(), 28, Color.white);
WordHuntGame.Ui.cs:497 / void BuildBoosterBar(Rect r)	v.Count = HzUI.Label("N", badge.transform, "0", 32, Ink); v.Count.rectTransform.Stretch();

`WordHuntGame.Ui.cs:503 / void BuildBoosterBar(Rect r)`	`var lt = HzUI.Label("Lv", lk.transform, "CAMP " + d.Unlock, 26, Gold);`
`WordHuntGame.Ui.cs:524 / void RefreshBoosters()`	`v.Count.text = n > 0 ? n.ToString() : "+";`
`WordHuntGame.Ui.cs:544 / void BuildCoach()`	`_coachText = HzUI.Label("T", _coach, "", 36, Color.white, TextAnchor.MiddleLeft, false);`
`WordHuntGame.Ui.cs:644 / void ShowPauseVeil()`	`var veil = HzUI.Button("PauseVeil", _layer, null, new Color(0.02f, 0.1f, 0.07f, 0.6f), Color.white, TogglePause, 40, 0);`
`WordHuntGame.Ui.cs:650 / void ShowPauseVeil()`	`var t = HzUI.Label("P", _pauseVeil, "PAUSED", 110, Color.white); t.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(900, 140));`
`WordHuntGame.Ui.cs:652 / void ShowPauseVeil()`	`var s = HzUI.Label("S", _pauseVeil, "Tap anywhere to continue", 44, new Color(1, 1, 1, 0.85f), TextAnchor.MiddleCenter, false);`
`WordHuntGame.Ui.cs:698 / void HidePauseVeil()`	`var t = HzUI.Label("Title", ribbon.transform, title, 66, Color.white); t.rectTransform.Stretch(6);`
`WordHuntGame.Ui.cs:721 / void HidePauseVeil()`	`var b = HzUI.Label("Body", card.transform, body, 40, HzUI.Hex("#5A4636"), TextAnchor.MiddleCenter, false);`
`WordHuntGame.Ui.cs:803 / void Box(string label, string value, float x, Color col)`	`var v = HzUI.Label("V", bx.transform, value, 62, col); v.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(320, 80), new Vector2(0, 16));`
`WordHuntGame.Ui.cs:804 / void Box(string label, string value, float x, Color col)`	`var l = HzUI.Label("L", bx.transform, label, 28, new Color(1, 1, 1, 0.65f)); l.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(320, 40), new Vector2(0, -42));`
`WordHuntGame.Ui.cs:812 / void Box(string label, string value, float x, Color col)`	`var nt = HzUI.Label("T", nb.transform, "NEW BEST!", 34, Color.white); nt.rectTransform.Stretch();`
`WordHuntGame.Ui.cs:819 / void Box(string label, string value, float x, Color col)`	`var bw = HzUI.Label("Bonus", d.Extra, "+ bonus word " + _p.Bonus.Text, 32, HzUI.Hex("#B8860B"));`
`WordHuntGame.Ui.cs:848 / void ShowSettingsDialog()`	`MakeToggle(d.Extra, new Vector2(-170, 0), IcoVolume, IcoVolumeOff, "Sound", () => !HzAudio.Muted, on => HzAudio.Muted = !on);`

`WordHuntGame.Ui.cs:849 / void ShowSettingsDialog()`	`MakeToggle(d.Extra, new Vector2(170, 0), IcoMusic, IcoMusicOff, "Music", () => !HzAudio.MusicMuted, on => HzAudio.MusicMuted = !on);`
`WordHuntGame.Ui.cs:860 / void Refresh()`	`text.text = label + (on ? " ON" : " OFF");`
`WordHuntGame.Ui.cs:866 / void Refresh()`	`text = HzUI.Label("L", b.transform, label, 34, Color.white);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/wordhunt through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Music is Resources/wordhunt/Music/jungle_loop.ogg. Preserve its load key when replacing it, check loop boundaries/compression and test Sound and Music toggles separately. Kit UI uses supplied fonts; validate glyph coverage for new languages and retain the actual font licence files.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed puzzle, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.wordhunt`, version, scene orientation and signing. The build command writes `Builds/Android/wordhunt.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/wordhunt/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.